

Recurrent neural networks for CenterTask: architecture and design decisions

Paola Castillo

September 2026

Abstract

This document specifies the two recurrent neural networks used to model the behaviour of a rhesus monkey in a length-categorization task. Network 1 is a small trial-level network (“tiny RNN”, in the sense of Ji-An et al.) that predicts, trial by trial, the category chosen and its timing: it receives the previous choice, the previous bar length, the previous outcome, the current bar length and the rule cue, keeps a hidden state of one to four variables, and reads the choice out through two decision criteria on the length axis. It is trained by maximum likelihood of the animal’s choices and compared, at matched number of state variables, with hand-written cognitive models (model-free reinforcement learning, model-based reinforcement learning, a Bayesian observer and a criterion-shift model). Network 2 is a gated network, at a 10 ms step, that turns network 1’s trial-level read-out into the cursor trajectory. Every decision below is tied to what the task exports and to what the first sessions show. Sections 1–14 concern network 1; Section 15 specifies network 2; Section 16 ties the two together.

Contents

1	Purpose and scope	2
2	Why two networks, and what each one reads	3
3	A question this design cannot resolve	4
4	The task and its outputs	5
4.1	Trial structure	5
4.2	What the task writes	5
4.3	What the data show	7
5	Problem formulation	8
6	Type of network, size, and data budget	9
6.1	Which kind of recurrent network	9
6.2	What d is	10
6.3	What a hidden unit is here	10
6.4	Activation functions, and why those	10
6.5	Size of the network and of the data	10
7	Architecture	11
7.1	Overview	11
7.2	Inputs	11
7.3	State: a gated recurrent unit	12
7.4	Read-out: ordinal with two criteria	16
7.5	Activation functions	16
7.6	Learned parameters	16

8	Two ways of knowing the rule	17
9	Timing observables: decision time, execution time and movement takeoff	17
9.1	The previous trial's timing as inputs	18
9.2	The current trial's timing as outputs	18
9.3	Elapsed time as an input	19
10	Training	20
10.1	Gradient clipping	20
10.2	Data split: nested cross-validation by session	21
11	Comparison models	22
11.1	Formal definitions	22
11.2	Decision rule	23
12	Interpretation	23
13	Predictions	24
14	Data requirements and changes to the task	24
15	Network 2: the motor (joystick) network	25
15.1	Question and data	25
15.2	Model	25
15.3	Loss	26
15.4	Two training regimes	27
15.5	What is measured	27
16	Coupling: how the two networks fit together	27
16.1	Order and the channel	27
16.2	What can network 1's hidden state plausibly detect, given weak sequential effects? .	28
16.3	Consequence: what this predicts for network 2	29
16.4	Changes to the export and to the task code	30
A	Model skeleton in PyTorch	31

1 Purpose and scope

Four questions are asked of this design, and for each one the answer is a specific, pre-registered procedure, not an impression from looking at fitted curves. A fifth question, of the same shape, is stated separately in Section 3 because it cannot be answered by this design at all.

- How much does the past weigh?** The categorization rule is fixed and the reward is a deterministic function of the stimulus, so no previous trial carries information about the correct answer to the current one. Any influence of the past is a bias. Does it exist, how large is it, and what shape does it have?
How this is resolved. By a chain of nested comparisons in held-out log-likelihood (Section 11): first M0 (no memory) against M1 (one trial of linear history) — if M1 does not beat M0, the past does not weigh at all; then M1 against the tiny RNN at $d = 1, 2, 3, 4$ — the smallest d that matches or beats M1 is the answer to “how large”, measured in state variables rather than trials. “What shape” is answered afterwards, by the diagnostics of Section 12 (criterion trajectories, phase portrait, gate statistics), never by the NLL comparison alone.
- What is the form of the update rule?** Compared with a criterion model written by hand (M2, Eq. (18)), with the same number of state variables, does the network find a dynamics the hand-written model cannot express?

How this is resolved. M2 and the RNN are fitted on the same folds (Section 10.2) with $d = 2$ for both, and compared by held-out NLL, paired by session, with a bootstrap interval on the difference (Section 11, “Decision rule”). A tie means M2’s formula is already correct. A win for the RNN means the formula is wrong in a way that the closed-form fit of Section 12, item 6, is meant to name.

3. **How does the 2- versus 3-category rule enter?** All at once, through the cue, or gradually, through feedback?

How this is resolved. Two versions of the same network are fitted, one with the cue as an input channel and one without it (Section 8), and their fitted criteria are compared trial by trial around every block switch in the **alternate** sessions. A jump on the switch trial supports the cue-driven version; a multi-trial drift supports the latent one; the two are not mutually exclusive; the cue-driven version’s own residual drift (if the cue-driven fit still shows a slower-than-instant adjustment) is itself the answer to whether the animal fully uses the cue.

4. **Does the decision state explain the movement?** The choice and its timing are one thing; the actual joystick trajectory that carries them out is another. Does knowing $\mathbf{h}_t$ change the prediction of the trajectory, beyond knowing only which target the animal walked to?

How this is resolved. This is network 2’s question (Section 15), with two nested controls (Section 15, “What is measured”): $\mathbf{b}$ versus nothing, and $\mathbf{b}$ versus the single scalar distance-to-criterion D_t it is built to summarise. Section 16 explains why the first control is expected to matter *even under weak trial-to-trial memory*, because most of $\mathbf{b}$ ’s content is a same-trial, not a cross-trial, quantity.

What network 1 does *not* do on its own: it has no time resolution finer than a trial, so it cannot say *when within the 1.5 s of a trial* the category is settled, only *how long*, as a trial-level average (Section 9). Modelling the continuous within-trial formation of the decision would need a third, separate network, at a 10 ms step; this design deliberately does not include one (Section 2, last paragraph, states what is given up by that choice).

2 Why two networks, and what each one reads

The project has two recurrent networks because the behaviour has two time scales. Across trials, the question is how the choice on one trial depends on the trials before it, and how quickly and accurately it is carried out. Inside a trial, the question is how that decision becomes a joystick movement. Each of the two has its own observable to explain: the choice with its timing, and the cursor trajectory. Each therefore reads a different file and a different set of columns (Table 1; the same two are drawn unrolled in Figure 9).

Network 1 is specified in Sections 5–14, network 2 in Section 15, and the link between them in Section 16.

Table 1: The two networks: what each one is for, what it reads, and what it predicts. Column names are those of the task’s export files.

	1. Trial-level tiny RNN	2. Motor (joystick) network
Question	How does the choice on trial t , and its timing, depend on the trials before it and on the rule cue?	Does trial 1’s decision state explain the cursor trajectory, beyond the chosen target and the go signal?
Specified in	Sections 5–14	Section 15
One step is	one trial	10 ms
One sequence is	one session (240–465 trials)	one trial (≈ 335 steps, bar onset -300 ms to go $+1500$ ms)
File read	<code>trial_data_<runTag>.csv</code> (one row per trial), plus one derived column per trial (takeoff heading) that the dataset notebook adds from the trajectory	<code>trajectory_<runTag>.csv</code> (one row per cursor sample, with <code>Time_ms</code> and <code>Epoch</code>) joined to <code>trial_data</code> on (<code>Block</code> , <code>TrialNumInBlock</code> , <code>Attempt</code>)
Input features	<code>BarSizeVA_deg</code> of rows t and $t - 1$; <code>ChosenTarget</code> , <code>IsCorrect</code> of row $t - 1$; <code>NumCategories</code> of row t ; optionally <code>DecisionTime_s</code> , <code>ExecutionTime_s</code> and the takeoff heading of row $t - 1$	$\mathbf{b}$, a low-dimensional read-out of network 1’s own $\mathbf{h}_t$ on <i>this</i> trial, held constant across the trial’s steps; <code>DirectionChosen</code> as a one-hot switched on at go; the go channel
Target	<code>ChosenTarget</code> of row t ; optionally <code>DecisionTime_s</code> , <code>ExecutionTime_s</code> and the takeoff heading of row t	<code>X_px</code> , <code>Y_px</code> at every step, centre-relative and divided by the 406 px centre-to-target distance; takeoff time, takeoff heading and <code>ExecutionTime_s</code> are read from the predicted trajectory as checks against network 1’s timing heads
Loss	cross-entropy of the choice; optional Gaussian terms on log decision time, log execution time and takeoff heading	squared error on position, valid steps only
What is <i>not</i> used	the trajectory samples themselves; the takeoff heading reaches it as one number per trial, precomputed by the dataset notebook	the bar length directly (it arrives only through $\mathbf{b}$); the previous trials (a trial is a complete sequence on its own)

Two consequences of this split are worth stating once. First, network 1 works on one row per trial: everything it needs is in the trial table, and the one quantity that comes from the trajectory (the takeoff heading) is reduced to a single number per trial before it reaches the network. Second, network 2 never looks across trials: a trial is a complete sequence for it, and the only way history reaches it is through $\mathbf{b}$, which is entirely determined by network 1’s read-out on that one trial. Network 2 is not optional plumbing on top of network 1; $\mathbf{b}$ is the sole channel through which trial identity and decision state reach the movement, and Section 16 returns to exactly what that channel carries and what it cannot.

What a third network would have added, and why it is not here. An earlier version of this design included a third, continuous-time network operating within the trial (10 ms step) whose job was to read the category out *as it forms*, giving a mechanistic account of *when* within the 1.5 s from bar to go the decision is settled, and admitting a biologically constrained excitatory/inhibitory variant comparable to extracellular recordings. Dropping it is a deliberate scope decision: those two questions (continuous within-trial dynamics, single-unit circuit mechanism) are set aside, and what survives of the timing question is answered at the trial level instead, as a single number per trial (decision time, Section 9) rather than a continuous curve.

3 A question this design cannot resolve

Whether the animal categorises by comparing the bar to a *boundary* (the view built into every equation above) or by comparing it to a *prototype* of each category and picking the most similar

one is a real, classical question in categorization research, and this design cannot answer it from the data it uses. The reason is not a lack of data; it is an identifiability result.

Why not. On a single ordered continuum, a boundary rule and a prototype/similarity rule with equal-variance categories make *identical* choice predictions: deciding by “which side of c_1 ” is mathematically the same computation as deciding by “which of the two nearest category means is closer”, because the equidistant point between two equal-variance Gaussian prototypes *is* the boundary. No amount of choice data, however much of it, can tell the two apart when this holds. The read-out of Section 7.4 is a boundary model *by construction* — it has two criteria c_1, c_2 and nothing else — so even the flexible GRU behind it cannot discover a prototype-based strategy: the limitation is in the read-out’s functional form, not in d or in the amount of training data.

What would make it resolvable, and why it is out of scope here. A genuine test requires either (a) an alternative, similarity-based read-out ($P(a_t = k) \propto \exp(-(\hat{x}_t - \mu_k)^2/2\sigma^2)$ for three learned prototypes μ_k) fitted alongside the ordinal one and compared by held-out NLL — informative only if the two turn out *not* to tie, which the equivalence result above says they will whenever the underlying category variances are equal — or (b) a categorical-perception discrimination sub-task (same/different judgements on pairs of bars, some crossing a category boundary, some not, matched in physical distance), which is the standard way the literature dissociates the two theories: a boundary account predicts sharpened discrimination exactly at the boundary; a prototype account does not. CenterTask has no such discrimination trial, so (b) is not answerable with the data this design uses, under any amount of modelling.

What is partially informative, and already available. The one asymmetry the existing data can speak to is at the edge of the Short category, which has no lower boundary (4.00 deg VA is the edge of the stimulus set, not a criterion). A boundary account predicts 4.00 is the easiest, fastest Short trial, since it is furthest from the only boundary that constrains Short. A prototype account does not have to predict this, since 4.00 could be an atypical member of Short if the category’s prototype sits nearer its centre. Comparing decision time (Section 9) across the four Short lengths — flat, or graded with distance to the category’s own centre — is a cheap, already-planned analysis (it reuses the chronometric check of Section 12, item 7) that bears on this question without resolving it outright.

4 The task and its outputs

4.1 Trial structure

Each trial of CenterTask (`CenterOutTask.m`) shows a horizontal bar of one of twelve lengths (4.0–7.8 degrees of visual angle), then a colour cue, then two or three coloured targets at cardinal positions that are reshuffled on every trial. The subject moves a joystick cursor from the centre to the target whose colour matches the bar’s category and is rewarded if the category is correct. Figure 1 shows the category structure; Figure 2 the on-screen sequence.

4.2 What the task writes

For every session the task writes `trial_data_<runTag>.csv` (one row per trial), two trajectory exports at the cursor sampling rate (about 9 ms), a per-condition summary and a text report. Only the trial table is used here. Its relevant columns are listed in Table 2.

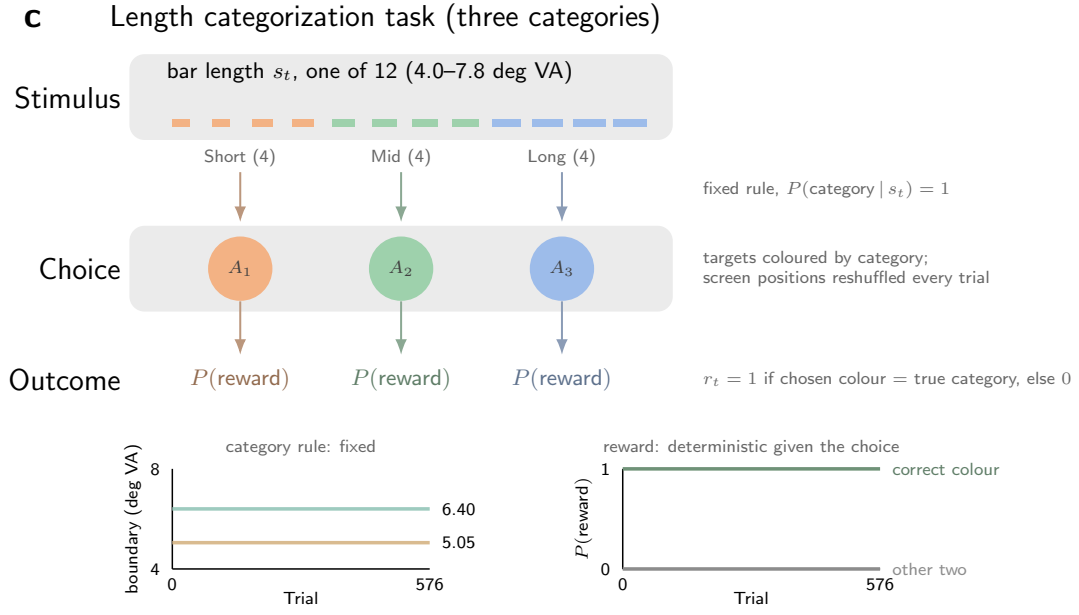


Figure 1: Task structure in the three-category framing. Twelve bar lengths, four per category, map deterministically onto Short, Mid and Long. The category boundaries (5.05 and 6.40 deg VA) are the nominal midpoints between adjacent lengths of different categories; they never change within or across sessions. Reward is deterministic given the choice. In the two-category framing (session mode `2cat`) the split is lengths 1–6 versus 7–12, with a single boundary at 5.85, and only the Short and Long targets are shown.

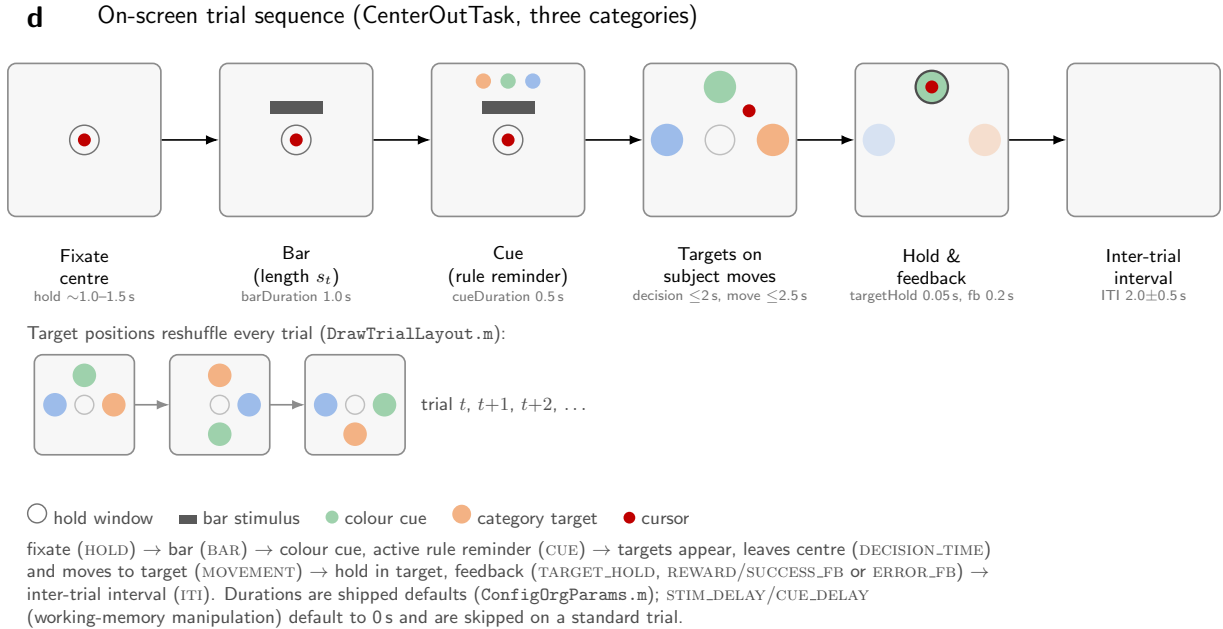


Figure 2: On-screen sequence of one trial, following the epoch order in `TaskEpoch.m`: fixation at the centre, bar (1.0 s), colour cue (0.5 s, two or three dots that remind the subject of the active rule), target onset (go), movement, hold and feedback, inter-trial interval. The bar-to-go interval is essentially fixed (1.55 s, range 35 ms). Target positions reshuffle every trial, so repeating a category and repeating a direction are dissociable.

Table 2: Columns of `trial_data_*.csv` used by the model and their encoding.

Model variable	Column	Encoding
s_t	BarSizeVA_deg	scalar, normalised to $[-1, 1]$ over 4.0–7.8
a_t	ChosenTarget (1 Short, 2 Mid, 3 Long, 0 none)	ordinal label 0/1/2; none $\rightarrow$ masked
r_t	IsCorrect	binary
cue_t	NumCategories (2 or 3; absent in old schema $\rightarrow$ 3)	one-hot, 2 channels
order	Block, TrialNumInBlock	defines the sequence within a session
retry	Attempt	only Attempt = 1 is kept

The history inputs are the same variables shifted by one trial: a_{t-1} , s_{t-1} , r_{t-1} . The first trial of a session receives a “no history” vector (zeros plus an indicator channel), never the last trial of the previous session.

4.3 What the data show

Thirteen sessions (20 August to 5 September 2026, subject ROM) hold 3 866 trials, of which 3 818 have a response. Overall accuracy is 0.765 and is stable across sessions (0.73–0.81). Errors are almost all wrong-target choices (860) rather than early exits (48). Retries are rare (51).

Psychometric function. Table 3 pools the three-category sessions. The subject’s Short/Mid boundary sits below the nominal 5.05: at 4.70 it already answers Mid half of the time. The Mid/Long boundary falls near the nominal 6.40. A network fitted to behaviour must reproduce this asymmetry; a network trained on the correct labels will not.

Table 3: Probability of each response as a function of bar length, three-category sessions pooled ($n \approx 310$ per length).

Length (deg VA)	$P(\text{Short})$	$P(\text{Mid})$	$P(\text{Long})$
4.00	0.98	0.01	0.01
4.10	0.96	0.02	0.02
4.70	0.42	0.52	0.07
5.00	0.23	0.66	0.11
5.10	0.15	0.75	0.10
5.70	0.04	0.79	0.17
6.00	0.03	0.78	0.19
6.10	0.02	0.68	0.30
6.70	0.02	0.19	0.79
7.00	0.00	0.09	0.90
7.60	0.01	0.02	0.97
7.80	0.02	0.03	0.96

Sequential effects are weak. Accuracy is 0.788 after a correct trial and 0.692 after an error ($n = 2953$ and 913). The probability of repeating the previous category is 0.322 after a correct trial and 0.293 after an error, against a chance level of $1/3$. The past therefore contributes little beyond a post-error effect. This bounds what the network can find and motivates the small d below.

The alternate regime is the base regime. From 5 September onward sessions run in session mode `alternate`: full 48-trial blocks of two and three categories follow each other (the first such session ran 2, 2, 3, 3, 2, 2, 3). Accuracy was 0.795 in the two-category blocks and 0.744 in the three-category blocks of that session. This is the regime the network is designed for: identical stimuli, a rule that changes within the day, signalled only by the cue, and a block switch every one to three blocks. The design below assumes a corpus of `alternate` sessions of the same size as the three-category corpus already recorded (12 to 15 sessions, 50 to 70 block switches; the budget is

derived in Section 6). The three-category-only sessions of August and early September are kept as auxiliary training data (Section 10.2).

Within-trial timing. Network 2 (Section 15) needs the durations of the epochs inside the trial, and network 1’s timing heads (Section 9) are checked against them. Table 4 gives them for one full trajectory export; the schedule from bar onset to go is essentially fixed, so one input schedule serves every trial, although the export uses each trial’s own onsets. The optional working-memory delays (STIM_DELAY, CUE_DELAY) are zero in every session recorded.

Table 4: Epoch durations inside the trial (session 31-Aug, 323 trials), from the Epoch column of the trajectory export.

Epoch	Median	5th–95th pct.
HOLD (centre)	992 ms	559–1 426
BAR	1 026 ms	1 009–1 026
CUE	509 ms	509–526
Bar onset → go (DECISION onset)	1 550 ms	1 533–1 568
DECISION (go → leave centre)	359 ms	292–492
MOVEMENT (leave → reach target)	176 ms	59–892
ITI	2 117 ms	1 567–4 016
Cursor sampling interval	9.05 ms	7.4–16.7

Screen geometry. Network 2 predicts position, so its units come from the rig (Table 5).

Table 5: Geometry, from ConfigOrgParams.m and CenterOutTask.m.

Hold window	200 px diameter
Targets	180 px diameter, at $320 \times 1.27 = 406$ px from the centre
Pixel pitch, viewing distance	0.3108 mm/px, 400 mm
Target eccentricity	12.6 cm, about 17.5°
Screen centre	(960, 540) px; X_px, Y_px are absolute

Filters. Table 6 lists what is removed and why. After filtering there are about 3 750 labelled trials in the 12 sessions recorded so far, 190 of them in two-category blocks; the **alternate** corpus grows with every new session.

Table 6: Exclusions.

Filter	Reason	Trials
Attempt > 1	retries re-show a stimulus the subject just failed and distort the psychometric function	51
ChosenTarget = 0	no choice to predict; the trial stays in the sequence as input to the next one but leaves the loss	48
Session 28-Aug	21 trials, accuracy 0.33	21

5 Problem formulation

Each session is one sequence. At step t the network receives $\mathbf{x}_t$, updates its state $\mathbf{h}_t$ and outputs $\hat{\mathbf{y}}_t = P(a_t \mid \mathbf{x}_{1:t})$. This is a many-to-many problem with $T_x = T_y$: one output per input step, with T equal to the number of trials in the session (240–465). It is neither many-to-one nor an encoder–decoder (Figure 3).

The objective is the likelihood of the observed choices,

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{lab}}} \sum_{\text{sessions}} \sum_{t=1}^T m_t \log P_{\theta}(a_t = a_t^{\text{obs}} | \mathbf{x}_{1:t}), \quad N_{\text{lab}} = \sum m_t, \quad (1)$$

where $m_t = 1$ if the trial has a response and `Attempt` = 1, and $m_t = 0$ otherwise. The network is not trained to solve the task; it is trained to predict what the subject did, errors included.

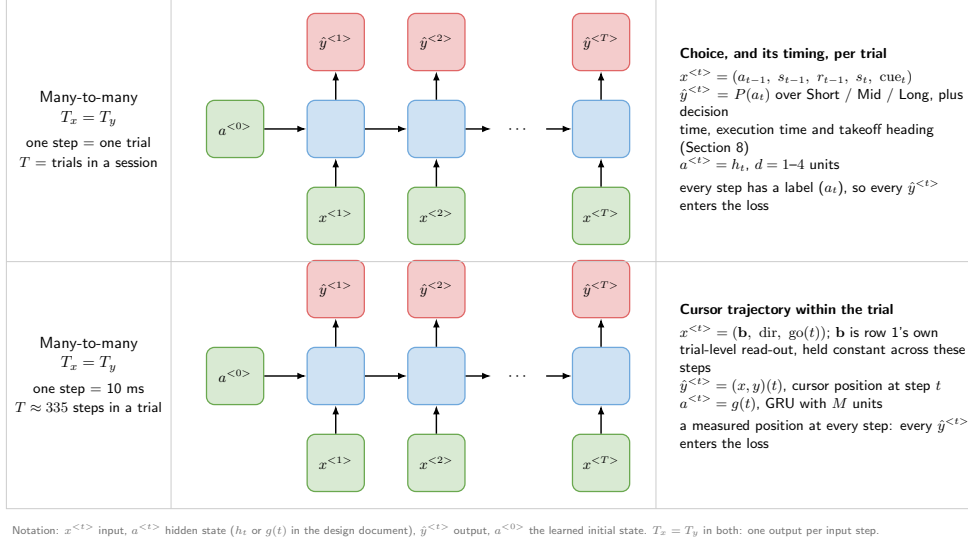


Figure 3: The two networks of the project in the standard many-to-many layout, $T_x = T_y$. Green boxes are inputs $x^{(t)}$, blue boxes the hidden state $a^{(t)}$ (our $\mathbf{h}_t$ or $g(t)$), red boxes the outputs $\hat{y}^{(t)}$, and $a^{(0)}$ the learned initial state. Top row: network 1, one step per trial, every step labelled. Bottom row: the motor network (Section 15), one step per 10 ms; none of its outputs are masked, since a cursor position is measured at every step.

6 Type of network, size, and data budget

This section answers, in plain terms, what kind of recurrent network is used and why, what its size parameter d means, which activation functions it contains, and how many sessions are needed. The formal equations follow in Section 7.

6.1 Which kind of recurrent network

A recurrent network is any model in which a state vector is updated at every step from its previous value and the current input. Three standard cells differ in how the update is written.

- **Vanilla (Elman) RNN.** One equation, $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$. Keeping a value and replacing it go through the same weights, so a unit cannot do both well, and gradients through hundreds of steps vanish or explode.
- **GRU (gated recurrent unit).** The same state, plus two gates that decide, per unit and per step, how much of the old value to keep and how much to overwrite (Eqs. (2)–(5)). One state vector, three weight sets.
- **LSTM.** Two state vectors (cell and hidden) and three gates. More parameters, and the “number of state variables” is ambiguous because each unit carries two numbers.

The network is a GRU, not a vanilla RNN. Three reasons. It is the cell used by Ji-An et al., so the comparison with their results is direct. With $d = 1$ or 2 the gates are what let a single unit hold a criterion across trials and still reset it after an error or a cue; a vanilla unit has to choose one behaviour. And it has one state vector, so d is unambiguous. The LSTM is not used because

it doubles the state for no gain at this size. A vanilla RNN with the same d is fitted as a control (Section 11): with short memory the gates may not be needed, and if the two cells tie, the vanilla one is preferred because its dynamics are easier to read.

6.2 What d is

d is the number of hidden units, which for a GRU equals the dimension of the state vector $\mathbf{h}_t \in \mathbb{R}^d$, which in turn equals the number of dynamical variables: the number of quantities the model carries from one trial to the next. In the framing of Ji-An et al. this number, not the parameter count, is the measure of model complexity, because it is what a cognitive model also has: the criterion-shift model M2 has $d = 2$ (two boundaries), the Bayesian observer $d = 4$ (mean and width of each boundary), a model-free learner $d = 36$ (one value per length and category). Two models with the same d can be compared on equal terms even if one has twenty parameters and the other eighty, because they remember the same amount and differ only in the form of the update.

With $d = 1$ the entire memory of the animal, as the model sees it, is a single number that moves up and down across trials. That is deliberately restrictive: it is the case in which the whole dynamics can be drawn on a page. The prediction for this task is that $d = 1$ or 2 is enough, because the rule is fixed and sequential effects are weak. $d = 3$ and 4 are fitted to confirm that they do not improve held-out likelihood.

6.3 What a hidden unit is here

A hidden unit is one coordinate of $\mathbf{h}_t$: a scalar that is read from the previous trial, updated with the current inputs, and passed on. It is not a neuron and it is not a feature detector. In this network a unit is expected to behave as a slowly varying bias, for instance the current displacement of a criterion from its resting value. Units have no assigned meaning before training; their meaning is established afterwards by reading how they move the criteria (Section 12). That is the difference between the h_t of the network and the V_t or c_t of a cognitive model, which are named before any data are seen.

6.4 Activation functions, and why those

There are three places where a nonlinearity sits, and only one of them is a choice.

- **Gates: logistic σ .** A gate is a proportion, how much to keep and how much to replace, so it must lie in $(0, 1)$. The logistic function is the standard map onto that interval. Fixed by the GRU definition.
- **Candidate state: tanh.** The state must be bounded, otherwise it can grow without limit over the 300 steps of a session, and it must be symmetric around zero so that a criterion can move in either direction. tanh gives both; a ReLU would give neither. Fixed by the GRU definition.
- **Read-out: logistic or cumulative Gaussian.** Each criterion turns a distance on the length axis into a probability, so the read-out is a sigmoid curve; it is the psychometric function of the model. Logistic and cumulative Gaussian are both used in psychophysics and differ only by a scale factor of about 1.7 in the slope. The logistic is the default; the Gaussian is used when an exact match with the notebook’s psychometric fits is wanted. This is the one choice.

No softmax is used: the ordinal read-out already yields three probabilities that sum to one, and a softmax would discard the ordering of the categories.

6.5 Size of the network and of the data

The network has d units, $k = 9$ (or 10) input channels and between 42 and 189 learned parameters (Table 14). Each session is one sequence of 240 to 465 steps, and gradients are propagated through the whole session. This is small: a fit takes seconds, and the whole grid of models, folds and seeds fits in an afternoon.

The data requirement is set by three needs, and the largest of them decides.

1. **Trials per parameter.** A rule of thumb for a well-conditioned maximum-likelihood fit is at least 20 labelled trials per parameter. With $d = 2$ and 85 parameters that is 1 700 trials, about six sessions. The three-category sessions already recorded cover this for the choice model alone.
2. **Outer folds.** The paired comparison between models uses one difference per test session. Ten differences are the minimum for a bootstrap interval that means anything; twelve to fifteen are comfortable.
3. **Block switches.** The rule questions are answered on the trials after each switch. With segment lengths drawn from one to three blocks and about 320 trials per session, a session holds $\approx 320/\bar{L} - 1$ switches, where $\bar{L}$ is the mean segment length in trials. With segments mostly of one block ($\bar{L} \approx 65$) that is four to five switches per session; with segments averaging two blocks it is two to three. To estimate the post-switch curve on five trials, in both directions (2→3 and 3→2), with a resolution of about one trial in the time constant, at least 50 to 60 switches are needed.

Need 3 dominates. **The target is 12 to 15 alternate sessions, with segment lengths drawn from $\{1, 2, 3\}$ blocks with weights about 0.6/0.3/0.1**, which yields four to five switches per session and 50 to 70 switches in total, and at the same time satisfies needs 1 and 2. Ten sessions is the floor below which the latent-rule version (Section 8) cannot be fitted with confidence; the cue-driven version and the choice-only questions are already answerable with the sessions recorded. The three-category-only sessions add to need 1 but not to needs 2 and 3.

7 Architecture

7.1 Overview

Figure 4 places the network next to a cognitive model in the framing of Ji-An et al.: both are dynamical systems with d state variables, the same inputs and the same output. They differ only in who writes the update rule. In the cognitive model the variables have a meaning fixed in advance (here, decision criteria c_t) and the update rule is written by the modeller; in the network the variables h_t have no assigned meaning and the update rule is learned.

7.2 Inputs

The input vector has $k = 9$ channels (Table 7).

Table 7: Input channels x_t .

Channel	Dim.	Content
s_t	1	current bar length, normalised
s_{t-1}	1	previous bar length, normalised
a_{t-1}	3	one-hot of the previously chosen category
r_{t-1}	1	1 if the previous choice was rewarded
cue_t	2	one-hot: two categories / three categories
first	1	1 on the first trial of a session
total k	9	
$\log \text{DT}_{t-1}, \log \text{ET}_{t-1}, \vartheta_{t-1} $ (optional)	3	previous decision time, execution time and takeoff heading, standardised; Section 9

Design decision: s_t enters twice. The current length is fed both to the recurrent cell and directly to the read-out (Section 7.4). With $d = 1$ or 2, if s_t entered only through the cell, the unit would be spent representing the stimulus and no memory would remain. The direct connection leaves the cell free to represent only what changes from trial to trial.

a

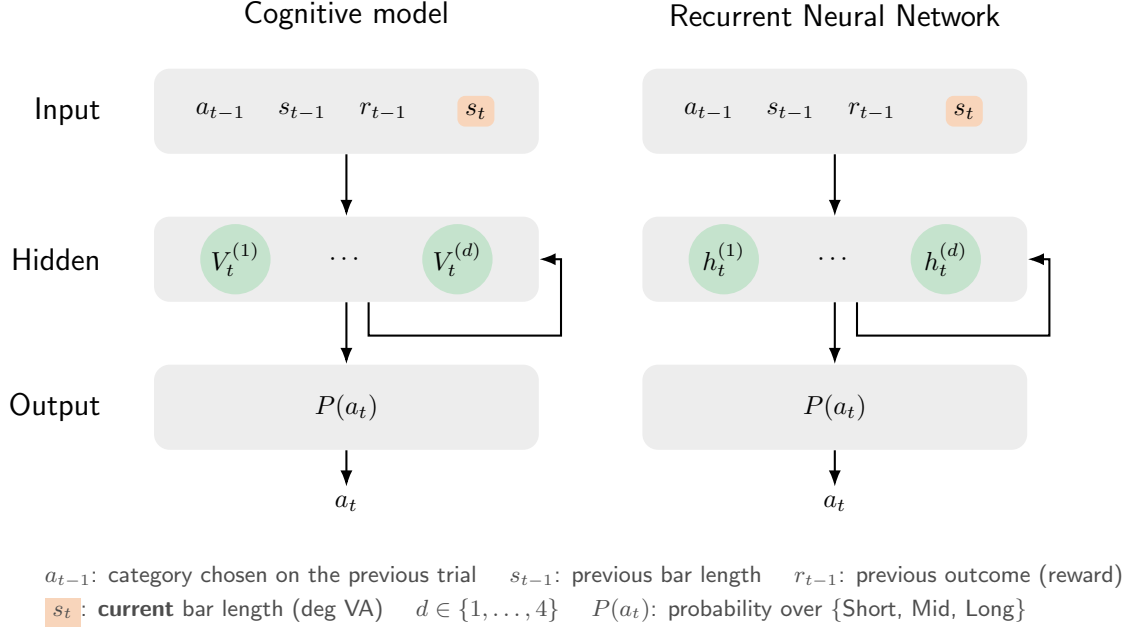


Figure 4: Trial-level model. Left: a cognitive model whose d state variables $V_t^{(i)}$ carry a meaning assigned by the modeller (for this task, decision criteria). Right: the recurrent network, whose d hidden units $h_t^{(i)}$ are interpreted after training. Both receive the previous choice a_{t-1} , the previous bar length s_{t-1} , the previous outcome r_{t-1} and, highlighted, the current bar length s_t , the input the reward-learning tasks of Ji-An et al. do not have. Both output a probability over Short, Mid and Long.

What is left out. PrevTrialDirection is logged but not used: without an export of the target layout it cannot be related to the chosen category. It becomes usable once the layout is exported (Section 14).

7.3 State: a gated recurrent unit

The state is updated by a GRU cell (Figure 5):

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{reset gate} \quad (2)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{update gate} \quad (3)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad \text{candidate} \quad (4)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{new state} \quad (5)$$

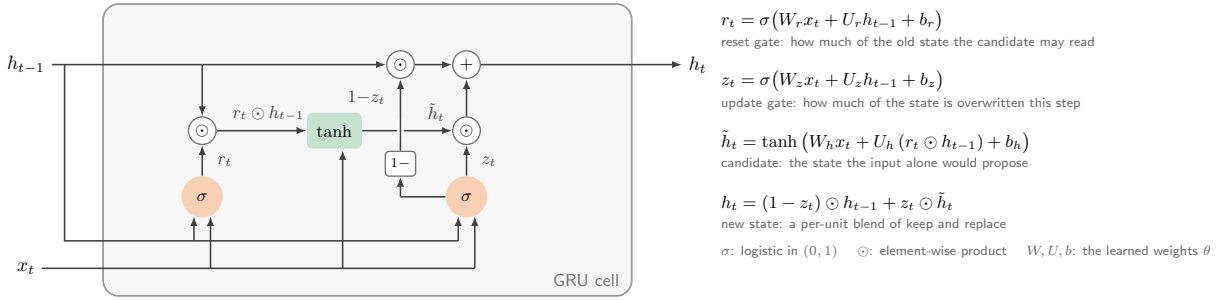
with σ the logistic function, $\odot$ the element-wise product, and $h_t \in \mathbb{R}^d$.

The two gates, step by step. A GRU unit does one thing per trial: it decides how much of what it remembered to keep and how much to replace with a fresh proposal. Read Eqs. (2)–(5) in the order in which the cell computes them.

1. The relevance (reset) gate r_t asks: is the past relevant to what I am about to propose?

It looks at the new input x_t and at the old state h_{t-1} and returns a number between 0 and 1 per unit. That number multiplies the old state *before* the candidate is computed (Eq. (4)). With $r_t = 1$ the candidate is built from the input and the full memory; with $r_t = 0$ the candidate is built from the input alone, as if the session had just started. It does not erase the memory; it only hides it from the proposal.

One GRU cell: two gates decide how much of the past survives



In the trial-level tiny RNN: $x_t = (a_{t-1}, s_{t-1}, r_{t-1}, s_t, \text{cue}_t)$ and h_t has $d = 1-4$ units. With $d = 1$ the cell has one reset value, one update value and one state, so the whole dynamics can be plotted.

What the gates mean here: $z_t \rightarrow 0$ keeps the criterion where it was (long memory, the Ji-An reward-learning regime); $z_t \rightarrow 1$ rewrites it from the current trial (short memory, the regime the weak sequential effects predict). $r_t \rightarrow 0$ lets the candidate ignore the past, which is how a rule switch can reset the state in one trial.

In the motor network: same cell with $x(t) = (b(t), \text{dir}, \text{go})$ every 10 ms and $M \approx 32$ units; the gates let it hold the pre-go position and then release the movement.

Why a GRU and not a plain RNN: a plain unit is $h_t = \tanh(W x_t + U h_{t-1} + b)$, so keeping a value and replacing it both go through the same weights; with one or two units it cannot do both well. The gates give each unit an explicit keep/replace switch that depends on the input, which is what lets a single unit hold a criterion across trials and still reset it after an error or a cue.

Cost: three weight sets instead of one. With $d = 2$ and five inputs that is about 50 parameters, still far below the 3800 trials available.

Figure 5: One GRU cell. The reset gate r_t decides how much of the old state the candidate may read; the update gate z_t decides how much of the state is overwritten; the new state is a per-unit blend of keep and replace. In this task, $z_t \rightarrow 0$ keeps a criterion where it was (long memory, the reward-learning regime of Ji-An et al.), $z_t \rightarrow 1$ rewrites it from the current trial (short memory, the regime the weak sequential effects predict), and $r_t \rightarrow 0$ lets the candidate ignore the past, which is how a rule switch can reset the state in a single trial.

2. **The candidate $\tilde{h}_t$ is the proposal.** It is what the unit would become if it listened only to this trial (and to whatever part of the past the relevance gate let through). It is a tanh, so it lives in $(-1, 1)$.
3. **The update gate z_t asks: how much of the proposal do I accept?** It also looks at x_t and h_{t-1} and returns a number between 0 and 1 per unit. Eq. (5) then mixes: a fraction z_t of the new value comes from the proposal, a fraction $1 - z_t$ is the old value carried over unchanged. With $z_t = 0$ the unit is frozen; with $z_t = 1$ it is overwritten.

The two gates answer different questions and can take any combination of values. $r_t = 0$ with $z_t = 1$ is a hard reset: the unit is overwritten by a proposal that ignored the past. $r_t = 1$ with z_t small is slow integration: the proposal uses the past and only a little of it is accepted each trial. $z_t = 0$ is pure memory, whatever r_t does. A vanilla unit has only the last equation with z_t fixed at one and r_t fixed at one, which is why it cannot both hold and reset.

In the words of this task. Take $d = 1$ and read the single unit as the displacement of the Short/Mid criterion from its resting value.

- On an ordinary correct trial the animal has no reason to move its criterion. The gates should give z_t small: keep what there is.
- After an error near the boundary, the criterion should shift towards the previous bar. The gates should give r_t high (the proposal starts from the current displacement) and z_t moderate (accept part of the shift). This is the criterion-shift rule of M2, Eq. (18), written as gate values.
- On the first trial after a block switch, the old criterion is useless. The gates should give r_t low and z_t high: propose from the cue alone and accept it. This is the reset that lets a rule switch act within one trial (Section 8).
- Because the gates are functions of the input, the same unit can do all three on different trials. That is what the network has to learn, and it is what the gate diagnostics of Section 12 read back out.

Notation across sources. The same cell appears under two sets of names. Table 8 maps them. Some introductory treatments first present a simplified GRU that has only the update gate and then add the relevance gate; the network here is the full cell, which is what `nn.GRUCell` implements and what Ji-An et al. use.

Table 8: Names of the GRU quantities in the deep-learning course notation and in this document.

Course notation	This document	Equation	Meaning
Γ_u , update gate	$\mathbf{z}_t$	(3)	how much of the state is rewritten
Γ_r , relevance gate	$\mathbf{r}_t$, reset gate	(2)	how much of the old state the candidate reads
$\tilde{c}^{(t)}$, candidate	$\tilde{\mathbf{h}}_t$	(4)	the state the input would propose
$c^{(t)} = a^{(t)}$	$\mathbf{h}_t$	(5)	the state passed to the next trial
$x^{(t)}$	$\mathbf{x}_t$	Table 7	the input of the trial
$a^{(0)}$	$\mathbf{h}_0$		the learned initial state

The final equation is the same in both notations up to the order of the terms: $c^{(t)} = \Gamma_u \odot \tilde{c}^{(t)} + (1 - \Gamma_u) \odot c^{(t-1)}$ is Eq. (5) with $\Gamma_u = \mathbf{z}_t$.

A worked numerical trace, $d = 1$. To make the four equations concrete, here is one hypothetical trial run through them by hand, with plausible trained parameter values plugged in directly at the level of gate *outputs* (the numbers a trained σ or $\tanh$ would return), so no weight matrices need to be guessed. Trial $t - 1$ was a Short response to a 5.00 bar, and it was wrong (5.00 is Mid); the criterion was sitting at $h_{t-1} = -0.10$.

Table 9: One hidden-unit update, traced step by step. “Value” is what the equation returns on this trial; “Reading” is what that number means for this task.

Quantity	Value	Reading
h_{t-1}	-0.10	the criterion’s displacement entering the trial
r_t (Eq. 2)	0.85	high: the candidate is allowed to read almost all of h_{t-1}
$r_t \odot h_{t-1}$	-0.085	the fraction of the old state the candidate actually sees
$\tilde{h}_t$ (Eq. 4)	-0.65	the proposal: given the input alone (plus that fraction of the past), the unit would become fairly negative — consistent with ‘push the criterion down, an error just happened at 5.00’
z_t (Eq. 3)	0.30	moderate: only three tenths of the proposal is accepted this trial
h_t (Eq. 5)	-0.265	the new state: $(1-0.30)(-0.10) + 0.30(-0.65) = -0.265$, a small step from -0.10 towards -0.65, not a jump to it

Two things to notice. First, r_t and z_t did different jobs: r_t controlled what the *proposal* was allowed to see, z_t controlled how much of that proposal was *kept*. Second, the actual update ($-0.10 \rightarrow -0.265$) is far smaller than the candidate (-0.65) because z_t was moderate; a network that instead learned $z_t \approx 0.9$ on this kind of trial would produce an update close to a full reset, and a network with $z_t \approx 0.05$ would barely move at all. Which of these the trained network actually does is an empirical question, read off exactly this way from real trials in Section 12.

What is fixed by the definition of a GRU, and what is a design choice here. These are two different lists, and confusing them is the usual source of “I don’t see what I could still change”. Table 10 keeps them apart.

Table 10: What cannot be changed without ceasing to be a GRU, versus what this project chose and could have chosen otherwise.

Fixed by the GRU definition	
Exactly two gates, $\mathbf{r}_t$ and $\mathbf{z}_t$, each a function of $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$	This is what makes it a GRU rather than a plain RNN (no gates) or an LSTM (a separate cell state and a third, output, gate)
Logistic σ in both gates	A gate is a per-unit proportion; it must live in $(0, 1)$, and σ is the standard map onto that interval
$\tanh$ in the candidate $\tilde{\mathbf{h}}_t$	The state must stay bounded over a 465-step session and be symmetric so a criterion can move either way; $\tanh$ gives both, a ReLU would give neither
The reset gate multiplies $\mathbf{h}_{t-1}$ specifically <i>inside</i> the candidate’s pre-activation (Eq. 4), nowhere else	This is the one place “the past” can be hidden from the proposal; it is not a free placement choice
The mixing equation $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$	This exact convex combination, with $\mathbf{z}_t$ playing both roles (weight on the new value, one minus the weight on the old one), is the GRU’s signature; an LSTM uses two independent gates here instead of one
A design choice for this project	
d , the number of units	Section 6; the main axis that is swept
What feeds $\mathbf{x}_t$ (the specific input channels, Table 7)	Has nothing to do with the cell being a GRU; any recurrent cell could take these same inputs
$\mathbf{h}_0$ (learned, shared)	Could instead be fixed at zero, or fitted per session
The read-out sitting on top of $\mathbf{h}_t$ (Section 7.4)	Not part of the GRU at all. This is the single biggest lever for changing what the model <i>can</i> express without touching the recurrent cell — it is also exactly what makes Section 3’s question unresolvable here: the read-out, not the cell, is what forces a boundary account
Regularisation (L2/L1 on the weights)	An external addition; the bare GRU has none
The plain-RNN comparison (Table below)	An ablation of the gating mechanism, not a modification of it

Table 11: Decisions about the recurrent cell.

Element	Decision	Justification
Cell type	GRU	Same as Ji-An et al., so the comparison is clean; with small d the gates let a single unit both hold and reset
Comparison	plain RNN $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$, same d	With short memory the advantage of gating may vanish; if the two tie, the plain cell is preferred for interpretability
d	1, 2, 3, 4, one at a time	The main complexity axis. Prediction: 1 or 2 suffice
$\mathbf{h}_0$	learned vector shared across sessions	The start of a session is the same “no history” state everywhere
Activations	σ in the gates, $\tanh$ in the candidate	Fixed by the GRU definition: proportions in $(0, 1)$, a bounded and symmetric state

7.4 Read-out: ordinal with two criteria

The categories are ordered along the length axis, so the output is parameterised as a perceived length and two criteria rather than three free logits:

$$\hat{x}_t = g(s_t, \mathbf{h}_t) = s_t + \mathbf{w}_g^\top \mathbf{h}_t + b_g \quad \text{perceived length} \quad (6)$$

$$c_1(\mathbf{h}_t) = c_1^0 + \mathbf{w}_1^\top \mathbf{h}_t \quad \text{first criterion} \quad (7)$$

$$c_2(\mathbf{h}_t) = c_1(\mathbf{h}_t) + \text{softplus}(\delta^0 + \mathbf{w}_2^\top \mathbf{h}_t) \quad \text{second criterion, } c_2 \geq c_1 \quad (8)$$

$$P(a_t \geq k) = (1 - \lambda) F(\beta(\hat{x}_t - c_k)) + \frac{\lambda}{2}, \quad k = 1, 2 \quad (9)$$

and the category probabilities follow from the cumulative ones:

$$P(a_t = \text{Short}) = 1 - P(a_t \geq 1), \quad P(a_t = \text{Mid}) = P(a_t \geq 1) - P(a_t \geq 2), \quad P(a_t = \text{Long}) = P(a_t \geq 2). \quad (10)$$

Table 12: Decisions about the read-out.

Element	Decision	Justification
Form	cumulative ordinal	Same structure as the criterion cognitive model and as the notebook’s psychometric fit, which uses the ordinal splits $P(\text{chosen} \geq k)$
F	logistic by default; cumulative Gaussian as an option	Logit and probit differ by a factor of about 1.7 in slope; the Gaussian matches the notebook exactly
β	learned slope, shared by both criteria	Perceptual sensitivity
λ	learned lapse, bounded in $[0, 0.1]$	Stimulus-independent errors; without it the slope absorbs the lapses
Order $c_2 \geq c_1$	softplus parameterisation, Eq. (8)	Guarantees a non-negative probability for Mid
Two-category blocks	same read-out, nothing masked	$P(\text{Mid}) = P(a \geq 1) - P(a \geq 2)$ vanishes when $c_1 \rightarrow c_2$: the network learns to collapse the criteria onto 5.85

Separation of biases. Whatever enters through g shifts *perception* and affects both rules equally (serial dependence on s_{t-1}). Whatever enters through c_k shifts the *decision* (post-error shift, rule). This separation is written into the architecture and is read directly from the weights $\mathbf{w}_g$ versus $\mathbf{w}_1, \mathbf{w}_2$.

7.5 Activation functions

No softmax is used: the ordinal read-out already produces a distribution that sums to one.

Table 13: Where each activation function sits and whether it is a choice.

Place	Function	Range	Chosen or fixed
Gates $\mathbf{r}_t, \mathbf{z}_t$	logistic	$(0, 1)$	fixed (GRU)
Candidate $\tilde{\mathbf{h}}_t$	tanh	$(-1, 1)$	fixed (GRU)
Perceived length $\hat{x}_t$	linear	$\mathbb{R}$	fixed (a position on the axis)
Criteria c_k	linear	$\mathbb{R}$	fixed
Cumulative probability	logistic or cumulative Gaussian	$(0, 1)$	chosen

7.6 Learned parameters

With $k = 9$ input channels the parameter counts are those of Table 14. With about 3 750 labelled trials there are between 20 and 90 trials per parameter, enough for $d \leq 2$. The $d = 4$ configuration needs stronger regularisation and is included to confirm that it does not improve.

Table 14: Number of learned parameters.

Block	Formula	$d = 1$	$d = 2$	$d = 4$
GRU	$3(dk + d^2 + d)$	33	72	168
$\mathbf{h}_0$	d	1	2	4
Read-out g	$d + 1$	2	3	5
Criteria	$2(d + 1)$	4	6	10
β, λ	2	2	2	2
Total		42	85	189

8 Two ways of knowing the rule

The subject sees two or three dots in the cue and two or three targets on screen. Giving that information to the network is not cheating: it models a subject who reads the cue. Withholding it forces the network to infer the rule from feedback, which is what the networks of Ji-An et al. do in reversal tasks. Both versions are fitted (Table 15, Figure 6).

Table 15: The two rule versions.

Version	cue_t	Models	Distinctive prediction
Cue-driven (main)	input channel	a subject who reads the cue	criteria move on the trial of the block switch
Feedback-inferred	channel off	a subject who infers the rule from errors	criteria drift over several trials until they collapse onto 5.85

The cue-driven version is the ceiling: the latent version cannot beat it, and the question is how close it gets and with what delay. A third reading comes for free: if, even with the cue given, the learned dynamics show gradual carry-over, the subject does not fully use the cue.

9 Timing observables: decision time, execution time and movement takeoff

Every trial with a response yields, besides the category chosen, three measurements of *how* the response was produced (Table 16). The model uses all three, as inputs from the previous trial and as outputs on the current one.

Table 16: The three timing observables. The takeoff is the same event that ends the decision time (the transition from Epoch 6 to 7); what the trajectory adds is its direction.

Observable	Definition	Source	Typical value
Decision time DT_t	from target onset (go) to the cursor leaving the 200 px hold window	<code>DecisionTime_s</code>	351 ms (292–492)
Execution time ET_t	from leaving the hold window to entering the 180 px target	<code>ExecutionTime_s</code>	183 ms (59–892)
Movement takeoff	the moment the cursor leaves the hold window (its time is go + DT_t) and its heading ϑ_t : the direction of the cursor over the first 50 ms after leaving, measured as the signed angle to the chosen target’s direction	derived by the dataset notebook from <code>X_px</code> , <code>Y_px</code> around the onset of Epoch 7; written to the per-trial table as <code>TakeoffHeading_deg</code>	0° on a straight reach; large on a change of mind

Trial-level RNN: how the network knows the rule (2 vs 3 categories)

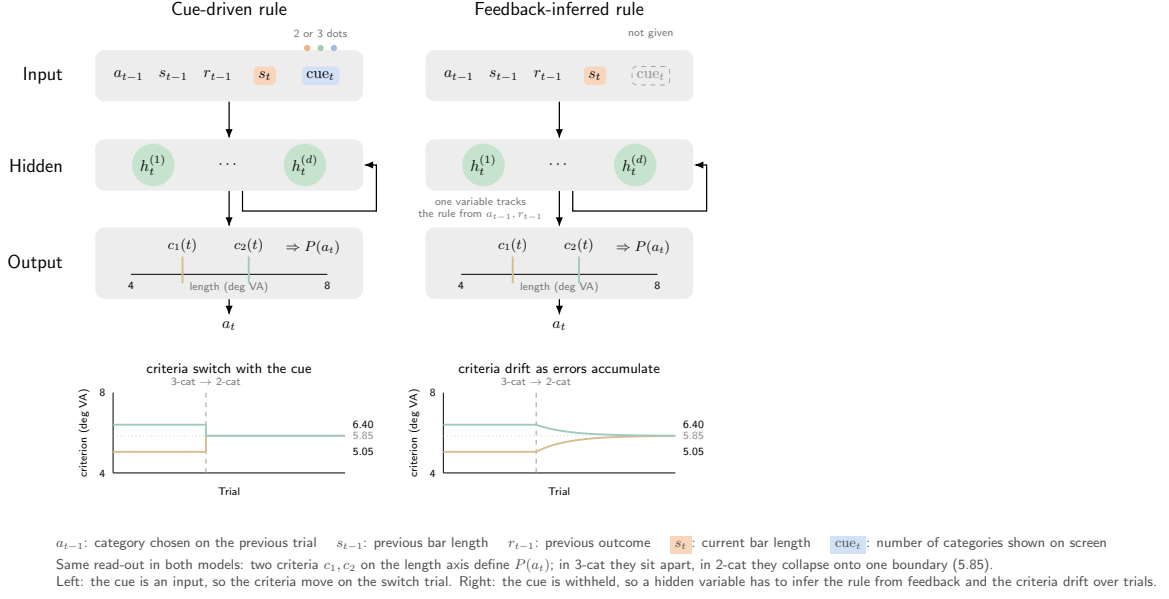


Figure 6: Cue-driven versus feedback-inferred rule. Both networks share the ordinal read-out with two criteria c_1, c_2 on the length axis. Left: the cue is an input, so the criteria can jump from the three-category positions (5.05, 6.40) to the two-category position (5.85) on the switch trial. Right: the cue is withheld, so a hidden variable must infer the rule from a_{t-1}, r_{t-1} and the criteria drift as errors accumulate. The lower plots are the predicted criterion trajectories around a block switch; once fitted, the same plot is produced from the data.

The three measure different things. The decision time is when the animal commits. The execution time is the vigour of the movement once committed. The takeoff heading is how committed the animal was at the moment of leaving: a reach that starts towards one target and bends towards another is a change of mind, and it shows as a heading error at takeoff before it shows in the endpoint. The takeoff heading is computed once by the dataset notebook and stored as a column of the per-trial table, so network 1 still reads one table with one row per trial.

9.1 The previous trial's timing as inputs

DT_{t-1} , ET_{t-1} and ϑ_{t-1} are observed before trial t , so they can be input channels without any leakage. They carry something a_{t-1} and r_{t-1} do not: how sure and how vigorous the subject was. A slow correct choice and a fast correct choice are different histories, and the criterion shift may depend on that. The channels are $\log DT_{t-1}$, $\log ET_{t-1}$ and $|\vartheta_{t-1}|$, each standardised within the training set and set to zero on trials without a response; the input dimension becomes $k = 12$. Each is tested with the same ablation as the other channels.

What cannot be done is to use DT_t , ET_t or ϑ_t , the current trial's values, as inputs for predicting a_t : they are part of the same response that is being predicted.

9.2 The current trial's timing as outputs

The more informative use is to predict the choice and the three timing observables jointly from the same state. Each gets a read-out from h_t and from the proximity of the perceived length to the

nearest criterion:

$$D_t = - \min_{k \in \{1,2\}} |\hat{x}_t - c_k(\mathbf{h}_t)| \quad \text{proximity to the nearest criterion} \quad (11)$$

$$\mu_t^{\text{DT}} = \mu_0^{\text{DT}} + \kappa_{\text{DT}} D_t + \mathbf{w}_{\text{DT}}^\top \mathbf{h}_t, \quad \log \text{DT}_t \sim \mathcal{N}(\mu_t^{\text{DT}}, \sigma_{\text{DT}}^2) \quad \text{decision time} \quad (12)$$

$$\mu_t^{\text{ET}} = \mu_0^{\text{ET}} + \kappa_{\text{ET}} D_t + \mathbf{w}_{\text{ET}}^\top \mathbf{h}_t, \quad \log \text{ET}_t \sim \mathcal{N}(\mu_t^{\text{ET}}, \sigma_{\text{ET}}^2) \quad \text{execution time} \quad (13)$$

$$\log \sigma_t^\vartheta = \nu_0 + \kappa_\vartheta D_t + \mathbf{w}_\vartheta^\top \mathbf{h}_t, \quad \vartheta_t \sim \mathcal{N}(0, (\sigma_t^\vartheta)^2) \quad \text{takeoff heading} \quad (14)$$

with all μ_0 , ν_0 , κ , $\mathbf{w}$ and σ learned. The decision time and the execution time are log-normal with a mean that moves with the state. The takeoff heading is zero-mean by construction (a straight reach) and what the state predicts is its *spread*: near a criterion, or after an error, the animal is expected to leave the centre less accurately.

The loss of Eq. (1) gains one term per observable,

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{choice}} + \lambda_{\text{DT}} \mathcal{L}_{\text{DT}} + \lambda_{\text{ET}} \mathcal{L}_{\text{ET}} + \lambda_\vartheta \mathcal{L}_\vartheta, \quad \mathcal{L}_{\text{DT}} = \frac{1}{N_{\text{lab}}} \sum m_t \left[\frac{(\log \text{DT}_t - \mu_t^{\text{DT}})^2}{2\sigma_{\text{DT}}^2} + \log \sigma_{\text{DT}} \right], \quad (15)$$

with $\mathcal{L}_{\text{ET}}$ of the same form and $\mathcal{L}_\vartheta = \frac{1}{N_{\text{lab}}} \sum m_t [\vartheta_t^2 / 2(\sigma_t^\vartheta)^2 + \log \sigma_t^\vartheta]$. The weights λ are chosen on the validation session so that no term dominates (start at 1 with every term in nats per trial). The same mask m_t applies: trials without a response have no timing.

Three reasons to do this.

1. **Four observations per trial instead of one.** With one categorical label per trial, the choice constrains the hidden state weakly. The three timing observables are continuous and constrain it far more. The joint fit is expected to identify d and the criterion dynamics with fewer sessions.
2. **A direct test of the criterion interpretation.** If c_1 and c_2 are the subject's boundaries, decision times must peak and takeoff headings must scatter where $\hat{x}_t$ falls near them. That is the chronometric function of the notebook (Section 1.6), computed with the model's boundaries instead of the nominal ones. Equations (12) and (14) make the test explicit: κ_{DT} and κ_ϑ should be positive. The execution time separates decision from vigour: if κ_{ET} is near zero while κ_{DT} is not, difficulty slows the decision and not the movement, which is the expected pattern.
3. **A link to network 2.** Network 2 (Section 15) produces a takeoff and an arrival as crossings of the hold radius and of the target when its predicted trajectory is simulated forward from $\mathbf{b}$ on a given trial; its simulated takeoff time, takeoff heading and execution time must agree with $\exp(\mu_t^{\text{DT}})$, ϑ_t and $\exp(\mu_t^{\text{ET}})$ respectively. This is the only place the two networks are cross-checked against each other, since network 2 has no internal read-out of the category itself.

To keep the comparison with the cognitive models fair, $\mathcal{L}_{\text{choice}}$ is always reported on its own, and the criterion model M2 receives the same three timing read-outs, Eqs. (11)–(14), which it can because it also has criteria. Models without criteria (M3) get the same read-outs with D_t replaced by the margin between the two largest $Q_t(s_t, \cdot)$.

9.3 Elapsed time as an input

The interval between trials is not constant: 2.0 s after a correct trial, 3.0 s after an error, plus pauses. If the memory of the criterion decays with real time rather than with trial count, a channel with Δt since the previous trial lets the GRU learn a time-dependent forgetting. The value comes from `Time_ms` in the trajectory export, not from the trial table. It is added last, and only if the ablation of r_{t-1} suggests decay.

Order of adoption. First the three previous-trial channels as inputs, because they are three more ablations and nothing else changes. Then the joint model with the three timing outputs, because that is where the scientific gain is. Then Δt , if needed.

10 Training

Table 17: Training protocol.

Element	Value	Justification
Loss	negative log-likelihood per trial, Eq. (1)	It is what is compared across models
Optimiser	Adam, learning rate 10^{-3}	Standard
Gradient clipping	global norm clipped at 1.0, before the Adam update	Gradients run through 240–465 steps; Section 10.1
Clipping diagnostic	fraction of steps on which the norm exceeded 1.0, logged per fit	Below 1 % the training is stable and clipping is only insurance; above that, lower the learning rate before raising the threshold
Batch	one whole session per sequence; batches of 4 sessions	The state must run through the session uncut, so that carry-over across blocks is observable
Early stopping	on validation NLL, patience 50 epochs	Replaces a fixed epoch count; prevents memorisation
Regularisation	L2 10^{-4} on all weights; L1 10^{-4} on $\mathbf{w}_g, \mathbf{w}_1, \mathbf{w}_2$	Pushes state-to-read-out connections that do not help to zero, which simplifies interpretation
Seeds	10 per configuration; best on validation reported, with the spread	With $d = 1$ the loss landscape has local optima
Distillation (optional)	teacher GRU $d = 16 \rightarrow$ student $d = 1, 2$ on the same sequences	Used by Ji-An et al. with scarce data; data are scarce here
Split	nested cross-validation by session, Section 10.2	The unit of independence is the day, not the trial
Report	mean test NLL per trial averaged over folds, bootstrap interval over sessions	

Trials without a response and retries remain in the sequence as input to the following trial; the mask $m_t = 0$ only removes them from the loss. Removing them from the sequence would break the history the subject actually experienced.

10.1 Gradient clipping

Gradient clipping is kept even though the network is tiny, for a reason that comes from the sequence length rather than from the size of the model.

Why the gradient can explode. The loss is propagated backwards through the whole session. The derivative of the loss at trial T with respect to the state at trial t contains the product of recurrent Jacobians

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{\tau=t+1}^T \frac{\partial \mathbf{h}_\tau}{\partial \mathbf{h}_{\tau-1}}, \quad (16)$$

a product of up to 465 factors. If the recurrent weights $\mathbf{U}$ have a gain above one in some direction, that product grows exponentially with $T - t$. The gates of the GRU address mainly the opposite problem, the vanishing of that product; against explosion they help less. The tanh bounds the state, not the gradient. With $d = 1$ or 2 an explosion is less likely than in a large network, but one exploding batch is enough to corrupt Adam’s running moments for many epochs afterwards.

Why clipping and not truncation. The classical alternative is truncated backpropagation, cutting the gradient at windows of, say, 50 trials. That would remove precisely the long-range dependence the model is meant to measure, the carry-over across block switches. Clipping, by contrast, changes neither the loss nor its optimum: it rescales the update only on the steps on which the global gradient norm exceeds the threshold, and is inert on every other step. Adam’s

per-parameter normalisation does not replace it, because that normalisation uses running averages which a single huge gradient biases.

Diagnostic. The global gradient norm is logged at every optimisation step. The fraction of steps on which it exceeded the threshold is reported with each fit (Table above). If that fraction is below 1 %, training is stable and clipping is only insurance. If it is higher, the learning rate is too high or the threshold too low, and the learning rate is lowered first; raising the threshold hides the problem. The teacher network used for distillation ($d = 16$) is the fit on which the diagnostic is most likely to fire.

10.2 Data split: nested cross-validation by session

Why the unit is the session. The state $\mathbf{h}_t$ runs through the whole session, so consecutive trials are dependent by construction. A day also has effects of its own that are not in the inputs: criterion drift, fatigue, joystick calibration. A split by trial would let the model learn the state of the day from the training trials and use it on the test trials of the same day, which inflates the likelihood and says nothing about generalisation. A split by trial is defensible with tens of thousands of trials per animal, as in the data of Ji-An et al.; it is not with about 300 trials per day.

Corpus. Two sets of sessions enter the fit. The **alternate** sessions are the target corpus: every question of Section 1 is answered on them. The three-category-only sessions are auxiliary: they enter training with $\text{cue}_t = (0, 1)$ throughout, add about 3 500 trials of psychometric structure, and are never used as test for any question that involves the rule.

Outer loop: leave one alternate session out. With S **alternate** sessions there are S folds. In each fold one **alternate** session is the test set and is not touched during fitting. The remaining $S - 1$ **alternate** sessions plus all auxiliary sessions form the fitting set. Test NLL per trial, averaged over folds, is the number reported for every model. This measures generalisation to a new day.

Inner loop: one validation session. Inside the fitting set, one **alternate** session is set aside as validation. It drives early stopping and the choice of regularisation strength and seed. It is rotated over the $S - 1$ candidates and the resulting models are averaged in NLL, or, if compute is short, fixed to the session closest in date to the test session. The number of state variables d is *not* selected in the inner loop: d is the quantity of interest, so every value is fitted separately and its outer test NLL is reported.

Paired comparison. All models of Section 11 are evaluated on the same folds. The difference in NLL between two models on each test session gives S paired values; the confidence interval is a bootstrap over those S differences. Day-to-day variation in absolute NLL cancels in the pairing.

Chronological check. In addition to the random leave-one-out, a forward-chaining pass: fit on all sessions before day j , test on day j , for every j from the third **alternate** session onward. If the chronological NLL is worse than the leave-one-out NLL, the subject is still changing between days and the report says so.

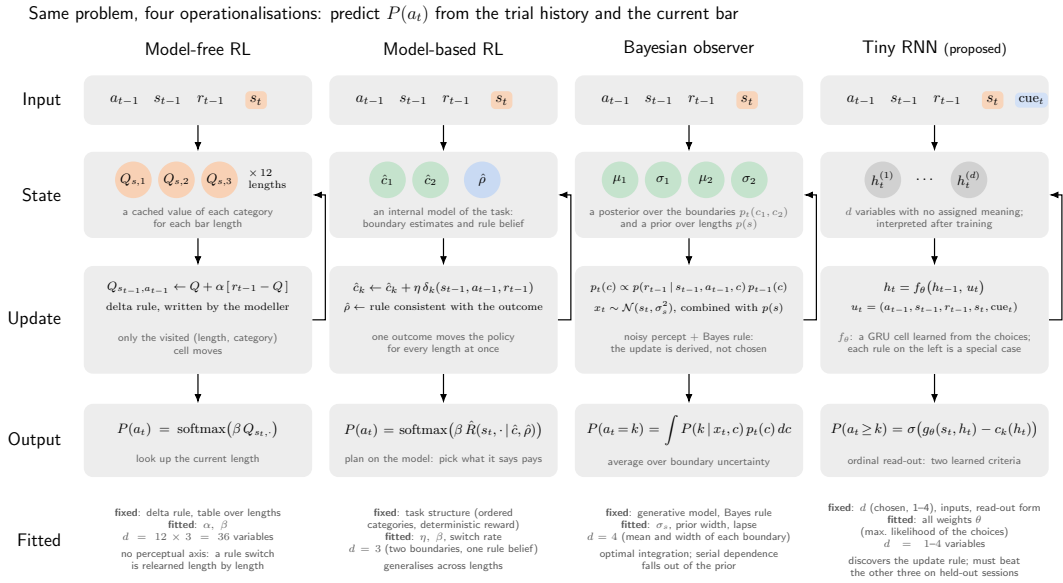
Secondary measure: blocks held out within a session. For the dynamics right after a block switch (Section 12, items 1 and 2) the outer folds are complemented by a within-session evaluation: in each **alternate** session, every second two-category block and every second three-category block is masked out of the loss while the network still reads it as input, and the NLL on the five trials after each masked switch is reported separately. This is the masked trial-level split, and it is legitimate here because the question is “within this day, does the model predict the trials that follow a switch?”. It is reported apart from the outer result, never mixed with it.

The day-specific component. The gap between the within-session NLL (blocks held out) and the between-session NLL (day held out) is a direct measure of what is specific to the day and not carried by the inputs. If it is small, the model transfers across days. If it is large, there is a state of the day the network cannot infer from the history, which motivates a per-session parameter or more sessions.

Cost. With $S = 10$ outer folds, four values of d , ten seeds and eight models the grid is about 3 200 fits. Each fit takes seconds on networks of this size.

11 Comparison models

All comparison models receive the same inputs, produce the same output and are fitted with the same loss and the same split. They are compared by held-out NLL, at matched d where applicable. Figure 7 shows how each one operationalises the same problem; Table 18 lists them.



All four are dynamical systems on the same inputs with the same output. They differ in what the state is, in who writes the update rule (the modeller, or the data) and how many variables d they carry. Following Ji-An et al., they are compared by held-out likelihood at matched d : a cognitive model that loses to an RNN with the same d has the wrong form of update, not too little memory.

Figure 7: Four operationalisations of the same problem. All four are dynamical systems on the same inputs with the same output. They differ in what the state is, in who writes the update rule (the modeller, a derivation, or the data) and in how many variables d they carry. Following Ji-An et al., they are compared by held-out likelihood at matched d : a cognitive model that loses to a network with the same d has the wrong form of update, not too little memory.

11.1 Formal definitions

M0, psychometric function without memory ($d = 0$). Equations (6)–(10) with $w_g = w_1 = w_2 = 0$: four parameters $c_1^0, \delta^0, \beta, \lambda$. Every gain is measured against this floor.

M1, one trial back ($d = 0$). M0 plus linear terms in the previous trial,

$$\hat{x}_t = s_t + \gamma_s s_{t-1} + \gamma_a^\top \text{onehot}(a_{t-1}) + \gamma_r r_{t-1} + \gamma_{ar} r_{t-1} \text{sign}(a_{t-1} - \bar{a}), \quad (17)$$

which separates “there is an effect of the previous trial” from “the effect accumulates”.

M2, criterion shift (the main cognitive model, $d = 2$). The two criteria are the state. After each trial they move towards the previous stimulus if the choice was wrong and decay towards a resting value:

$$c_{k,t} = c_{k,t-1} + \eta (1 - r_{t-1}) \text{sign}(s_{t-1} - c_{k,t-1}) \mathbb{I}[|s_{t-1} - c_{k,t-1}| < w] - \rho (c_{k,t-1} - c_k^0), \quad (18)$$

with learning rate η , window w and decay ρ , and the read-out of Eq. (9). This is the analogue of the “value” update of Ji-An et al., written for perceptual criteria.

M3, model-free reinforcement learning ($d = 36$). A table $Q_t(s, a)$ over the 12 lengths and 3 categories, updated by the delta rule $Q_t(s_{t-1}, a_{t-1}) = Q_{t-1}(s_{t-1}, a_{t-1}) + \alpha [r_{t-1} - Q_{t-1}(s_{t-1}, a_{t-1})]$ and read out by $P(a_t) = \text{softmax}(\beta Q_t(s_t, \cdot))$. It has no perceptual axis: a rule switch must be relearned length by length.

M4, Bayesian observer ($d = 4$). A Gaussian posterior over each criterion, $c_k \sim \mathcal{N}(\mu_k, \sigma_k^2)$, updated by Bayes’ rule with the likelihood of the observed outcome, $p_t(c) \propto p(r_{t-1} | s_{t-1}, a_{t-1}, c) p_{t-1}(c)$; a noisy percept $x_t \sim \mathcal{N}(s_t, \sigma_s^2)$ combined with a prior $p(s)$ over lengths; and a decision that averages over the posterior, $P(a_t = k) = \int P(k | x_t, c) p_t(c) dc$. The update is derived, not chosen; serial dependence falls out of the prior.

Table 18: Models compared. All share the inputs, the output and the loss.

Model	State	d	Establishes
M0 psychometric	none	0	floor without memory
M1 one trial back	none; previous-trial terms	0	effect versus accumulation
M2 criterion shift	c_1, c_2 with Eq. (18)	2	the hand-written analogue of value
M3 model-free RL	$Q(s, a)$ table	36	reference without a perceptual axis
M4 Bayesian	posterior mean and width per criterion	4	derived update
RNN-plain	plain RNN	1–4	whether gates are needed
RNN-GRU	this specification	1–4	
Shuffled	RNN-GRU with permuted labels		floor of the NLL

11.2 Decision rule

The network “discovers something” only if it beats M1 and M2 with equal or smaller d , in test NLL, with the bootstrap interval excluding zero. If it does not beat M1, the past acts for one trial and does not accumulate. If it beats M1 but not M2, the criterion model already captures the dynamics and is the model to report.

12 Interpretation

Once trained, the network is read on the stimulus axis, not on the state axis.

- 1. Criterion trajectories.** For every trial, $c_1(\mathbf{h}_t)$ and $c_2(\mathbf{h}_t)$ in degrees of visual angle, plotted against trial number with the block switches marked. This is the empirical version of the lower plots of Figure 6.
- 2. Phase portrait in criterion units.** With $d = 1$: $c_k(\mathbf{h}_t)$ against $c_k(\mathbf{h}_{t-1})$, one curve per input condition (correct/error $\times$ previous category). Where a curve crosses the diagonal there is a fixed point; its slope says how much is retained. This is the plot of Ji-An et al. transferred to length units.
- 3. Gates.** Distribution of z_t and r_t by condition. A mean z_t near 1 confirms short memory; r_t near 0 on the trials after a block switch confirms a reset.

4. **Input ablations.** Remove s_{t-1} , a_{t-1} , r_{t-1} or cue_t one at a time and measure the loss in NLL. This quantifies the three sources of the past: perceptual serial dependence, feedback-driven criterion shift, and rule.
5. **Perception versus decision.** Relative magnitude of w_g against w_1, w_2 : whether the past acts on perception or on the criterion.
6. **Closed form.** If $d = 1$, fit a simple expression (piecewise linear or a low-order polynomial) to the learned update curve. If it differs from Eq. (18), it is a new cognitive rule.
7. **Chronometric check.** In the joint model, observed decision time, execution time and absolute takeoff heading against $|\hat{x}_t - c_k(\mathbf{h}_t)|$ for the nearest criterion, binned. Decision time and heading spread must fall with distance (slopes κ_{DT} , κ_{ϑ}); execution time should not (Eqs. (12)–(14)).

13 Predictions

Table 19: Concrete predictions and their basis. Any of these that fails is a result, not a problem with the model.

Prediction	Basis
M1 beats M0 by a small margin: the post-error effect exists	accuracy 0.69 versus 0.79
RNN $d = 1$ matches or beats M1 by a small margin; $d = 2$ does not improve on $d = 1$ in three-category sessions	weak sequential effects
Mean $z_t > 0.5$: memory of one or two trials	fixed rule, deterministic reward
Fitted c_1 near 4.6, below the nominal 5.05; c_2 near 6.4	Table 3
On the five trials after each block switch the cue-driven version beats the latent one; the latent version closes the gap within about ten trials	the cue is visible; the split moves a boundary by 0.8 deg VA, which errors reveal quickly
The between-session NLL exceeds the within-session NLL by less than the gap between M0 and the best RNN	accuracy stable at 0.73–0.81 across days
Ablating s_{t-1} costs more NLL than ablating a_{t-1}	perceptual serial dependence is the typical history effect in categorization
In the joint model $\kappa_{DT} > 0$ and $\kappa_{\vartheta} > 0$: decision times peak and takeoff headings scatter near the fitted criteria	chronometric effect, notebook Section 1.6
$\kappa_{ET} \approx 0$: difficulty slows the decision, not the movement	execution time is motor; median 183 ms with a wide right tail unrelated to length
The joint model reaches the same choice NLL as the choice-only model with fewer sessions	the decision time is a continuous second observation per trial

14 Data requirements and changes to the task

The design assumes that **alternate** is the base session mode from now on. Table 20 lists what that corpus has to look like for the split of Section 10.2 to work, and what has to change in the task code.

Table 20: What the analysis needs from future sessions.

Need		Current state	What is required
Enough sessions	<code>alternate</code>	1 session, 6 switches	12 to 15 sessions, 50 to 70 switches in total (Section 6); ten is the floor for the latent-rule version
Variable segment length		fixed at one block (48 trials) per segment	draw <code>alternateBlocks2cat</code> and <code>alternateBlocks3cat</code> independently from $\{1, 2, 3\}$ with weights about 0.6/0.3/0.1 for every session, so the rule cannot be predicted by counting trials and switches stay frequent
Both switch directions		2→3 and 3→2 both present	keep alternating the starting rule across sessions so the two directions are balanced
A stable two-category split		marked “placeholder” in <code>ConfigBarLengths.m</code>	fix it: the model assumes c_1^{2cat} is the same in every session
Motor bias separated from decision bias		the target layout is not exported	export the category at each of the four positions; <code>PrevTrialDirection</code> then becomes an input channel
Uniform schema		old sessions lack <code>NumCategories</code>	the notebook’s normaliser already handles it; document it in the export

15 Network 2: the motor (joystick) network

15.1 Question and data

Network 2 asks how network 1’s decision state becomes a cursor trajectory, and whether that state explains the trial-to-trial variability of the movement (Section 1, question 4). One step is 10 ms; one sequence is one trial, from 300 ms before bar onset to 1 500 ms after go, about 335 steps (Table 4). Its data come from the full trajectory export (`trajectory_<runTag>.csv`, one row per cursor sample with `Time.ms` and `Epoch`) joined to the trial table on (`Block`, `TrialNumInBlock`, `Attempt`); the target is the measured cursor position, `X.px`, `Y.px`.

15.2 Model

$$\mathbf{b} = \mathbf{W}_b [\mathbf{h}_t; s_t] \quad \text{trial-level read-out, } k \text{ dimensions, one value per trial} \quad (19)$$

$$\mathbf{g}(t) = \text{GRU}([\mathbf{b}, \text{dir}_{\text{onehot}}, \text{go}(t)], \mathbf{g}(t - \Delta t)) \quad \text{Eqs. (2)–(5)} \quad (20)$$

$$\mathbf{p}(t) = \mathbf{W}_p \mathbf{g}(t) = (x, y)(t) \quad \text{cursor position, centre-relative} \quad (21)$$

$\mathbf{h}_t$ is network 1’s state *after* it has processed trial t (so it already depends on s_t through network 1’s own forward pass); $\mathbf{b}$ therefore carries the stimulus into the movement pathway only through the decision state, plus s_t itself, never through a separate direct sensory channel. Unlike a within-trial decision signal, $\mathbf{b}$ does not vary with t inside the trial: it is computed once, from network 1, and broadcast as a constant input to every step of network 2’s sequence (Figure 9).

Table 21: Decisions about the motor network.

Element	Decision	Justification
Source of $\mathbf{b}$	network 1's $\mathbf{h}_t$ (plus s_t), never a continuous within-trial signal	There is no within-trial decision network in this design (Section 2); $\mathbf{b}$ is the only channel by which trial identity reaches the movement
Bottleneck k	the number of components for 90% of the variance in the functional PCA of the movement curves, in the target frame; typically 3–5	It is the actual dimensionality of the movement shape, and the number to compare with the neural population later
GRU M	32 (sweep 16–64)	Two output dimensions, 3 800 trials
dir _{onehot}	DirectionChosen, not DirectionCorrect, on from go	It is where the animal went, also on error trials. The layout is random and independent of the stimulus, so the direction carries no information about the category, and it is logged on every trial with a response
Output	position , not velocity	Position is what is measured, every 9 ms. Velocity is derived and amplifies joystick noise
Units	pixels relative to the centre, divided by the 406 px centre-to-target distance	The target sits at radius 1 and the hold window edge at 0.25, the same scale throughout the trial

Why position and not velocity. Four reasons, stated once.

- The velocity in the export is obtained by differentiating the resampled position, and it needs a causal Butterworth filter at 10 Hz to be usable. That filter puts lag and design choices into the training target.
- The sampling interval jitters (median 9 ms, 95th percentile 16.7 ms); a finite difference with an irregular step is noisier still. Position does not suffer this.
- Movement onset and endpoint are defined in position by the task itself (leaving the 200 px hold window, entering the 180 px target), so the metrics are read directly from $\mathbf{p}(t)$ and coincide with `DecisionTime_s` and `ExecutionTime_s`.
- Predicting velocity and integrating it accumulates drift: every step can look right and the endpoint be wrong.

15.3 Loss

Squared error on $\mathbf{p}(t)$ over valid steps, with three terms.

$$\mathcal{L}_{\text{motor}} = \frac{1}{N_v} \sum_{t \text{ valid}} \left\| \mathbf{p}(t) - \mathbf{p}^{\text{obs}}(t) \right\|^2 + \gamma \sum_t \left\| \mathbf{p}(t) - \mathbf{p}(t - \Delta t) \right\|^2. \quad (22)$$

- **Before go** the target is the trial's actual hold position, not an exact zero: the cursor moves inside the 200 px window. The floor is the RMS of the position during HOLD, in pixels; a pre-go error below that RMS squared means nothing.
- **After go**, ordinary squared error up to the arrival at the target.
- **Smoothness**, the second term with a small γ , so that the network does not produce jumps. This recovers the one thing that a velocity target gave for free.

Curriculum: first the mean trajectories per length $\times$ direction, then single trials. The first step fixes the shape; the second asks whether trial-to-trial variability is explained from $\mathbf{b}$. Velocity, peak speed and curvature are computed afterwards, from the predicted and the observed trajectory with the same filter, and only as evaluation metrics.

15.4 Two training regimes

Regime	What is done	Purpose
Two-stage (first)	train network 1; freeze it; compute $\mathbf{b}$ from its frozen $\mathbf{h}_t$ on every trial; train $\mathbf{W}_b$ and the GRU on top	Interpretability: $\mathbf{b}$ is a read-out of a decision state fixed beforehand, not one that adapts to make the movement easier to fit
Joint	$\mathcal{L} = \mathcal{L}_{\text{choice}} + \lambda \mathcal{L}_{\text{motor}}$, gradients into both networks	Does the motor fit improve if network 1's representation is allowed to accommodate the movement?

15.5 What is measured

1. R^2 of position per step and per trial; endpoint error in pixels; takeoff-time error, with the takeoff defined as the first crossing of the hold radius, as in the task (so it is compared with `DecisionTime_s`); execution-time error, with arrival defined as entering the target (compared with `ExecutionTime_s`); takeoff-heading error over the first 50 ms after the crossing; and, derived from the trajectory, peak-speed and curvature error.
2. Against the linear decoder of the notebook at the same horizon: the GRU has to win, or the recurrence contributes nothing.
3. **Two nested controls**, both against the full model with $\mathbf{b}$: (i) the same GRU with direction and go only, no $\mathbf{b}$ at all — if this alone costs accuracy, *some* decision-related signal reaches the movement, which Section 16 argues is likely even with weak trial-to-trial memory; (ii) the same GRU with $\mathbf{b}$ replaced by the single scalar D_t (Eq. (11), already computed by network 1) instead of the full learned k -dimensional $\mathbf{b}$ — if (ii) matches the full model, $\mathbf{b}$ is not earning its keep beyond the obvious perceptual-distance signal; if the full $\mathbf{b}$ still wins, it carries something D_t alone does not (plausibly the rule identity, or a genuine cross-trial bias). Together, (i) and (ii) are the direct answer to question 4 of Section 1.
4. Error trials (`ErrorType` = 2): is the trajectory towards the wrong target predicted from $\mathbf{b}$ before movement onset? This links to the changes-of-mind analysis of the notebook.

16 Coupling: how the two networks fit together

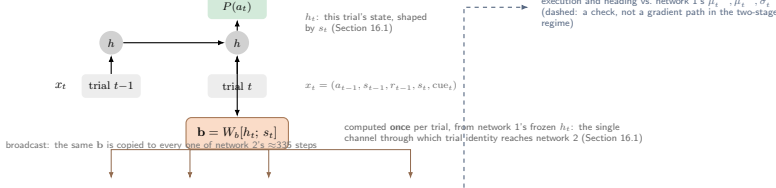
Figure 8 draws the coupling mechanism directly: network 1's read-out on trial t , reduced to a small vector, becomes a constant input to every one of network 2's ≈ 335 within-trial steps, and network 2's simulated trajectory is checked back against network 1's timing predictions. Figure 9 draws the same two networks in the generic many-to-many layout used throughout this document. With only two networks, the link between them is a single, mandatory channel rather than a set of optional cross-checks, and the rest of this section states precisely what that channel carries, why, and what follows for how $\mathbf{h}_t$ itself should be read.

16.1 Order and the channel

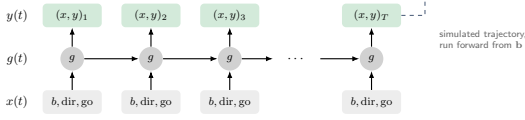
- **Order.** Network 1 is fitted first, on its own, using only `trial_data`. It is the cheapest, it fixes the minimal d of history, and it settles questions 1–3 of Section 1 before network 2 is touched. Network 2 is fitted second, in the two-stage regime and then jointly (Section 15).
- **The channel: $\mathbf{b}$.** Network 1's read-out $\mathbf{h}_t$ on trial t , linearly reduced to k dimensions together with s_t , is the *only* way trial identity and decision state reach network 2 (Eq. (19)). There is no separate route by which the stimulus, the history, or the rule can influence the predicted trajectory. This makes the nested controls of Section 15 (network 2 with $\mathbf{b}$ removed, or with $\mathbf{b}$ replaced by D_t alone) a clean test: whatever they cost is exactly what $\mathbf{b}$ was carrying, and the next subsection works out what that is likely to be.
- **Consistency of the timing.** Network 1 predicts a decision time, an execution time and a takeoff-heading spread as trial-level statistics (Section 9). Network 2, run forward with $\mathbf{b}$ from

Coupling: network 1's read-out on trial t becomes a constant input to network 2's within-trial loop

Network 1 (one step = one trial)



Network 2 (one step = 10 ms, same trial t)



Training order: network 1 is fitted first and frozen; b is computed from its frozen h_t for every trial of every session; network 2 is then fitted on top (two-stage regime), or the two are fine-tuned jointly afterwards (Section 15.4).

Figure 8: The coupling pipeline. Network 1 processes trial t and produces h_t ; b is computed once from h_t and s_t (Eq. (19)) and broadcast, unchanged, to every step of network 2's within-trial loop for that same trial. Network 2's simulated trajectory, run forward from b , is then checked back against network 1's timing predictions (dashed): this is a post-hoc consistency check, not a path gradients flow through in the two-stage training regime.

that same trial, produces a full simulated trajectory, from which the same three quantities can be measured directly (takeoff and arrival as crossings of the hold radius and the target). The two must agree, trial by trial; a systematic gap says one of the two networks carries a bias the other does not.

- **What this design gives up.** Because b is constant across a trial's 335 steps, network 2 has no way to represent *when within the trial* the decision was reached beyond what b already summarises; it can only act on a decision already made. A continuous within-trial account of decision formation is not available in this design (Section 2).

16.2 What can network 1's hidden state plausibly detect, given weak sequential effects?

Section 2's “What the data show” pools sequential effects that are small: accuracy 0.788 after a correct trial versus 0.692 after an error, and repeat-category probability within a few points of chance. It is tempting to conclude that h_t therefore has little to do, and that the whole coupling built around it is coupling to noise. That conclusion moves too fast, for a reason built into x_t itself: h_t is not only a memory of the past. It is computed *using* s_t , the current trial's own stimulus (Table 7), so it is shaped by something with no cross-trial component at all before any question of memory arises.

Three different things can live inside a low- d h_t , at three different timescales, and weak single-trial serial effects rule out at most one of them (Table 22).

Table 22: Three candidate contents of $\mathbf{h}_t$, and how the design already tells them apart.

Candidate content	Timescale	How it is told apart
Same-trial decision state: how far s_t sits from the current criteria, D_t (Eq. (11)), i.e. how easy or hard <i>this</i> trial’s categorisation is	zero trials (within trial t)	Present even if $z_t \approx 1$ always (no real memory): $\mathbf{h}_t$ still differs trial to trial simply because s_t does. Not weakened at all by weak sequential effects
Rule / block identity: which of the 2-/3-category regimes is active	tens to ~ 150 trials (a block)	A much <i>longer</i> -range dependency than single-trial serial effects, and not bounded by them; isolated by the cue-driven-versus-latent comparison (Section 8) and by ablating cue_t (Section 12, item 4)
Genuine single-trial memory: a criterion shift that outlives one trial and decays (M2, Eq. (18))	one to a few trials	This is the one component the weak sequential effects (0.788 vs. 0.692) directly bound to be small; it is what survives <i>after</i> removing the first two rows above, tested by comparing the tiny RNN against M1 and M2 at matched d (Section 11)

Reading the diagnostics of Section 12 correctly. Three outcomes are all consistent with weak single-trial sequential effects, and they are not the same finding.

1. If $d = 1$ or 2 does not beat M0 at all (Section 11’s decision rule), $\mathbf{h}_t$ has collapsed onto a redundant, same-trial function of s_t and cue_t : the gates will show z_t near 1 essentially unconditionally (nothing is ever protected across trials), and the network is, in effect, the psychometric function with extra steps. This is itself a reportable finding, already listed in Section 13.
2. If d beats M0 but the gain tracks rule identity as a near step function between blocks, rather than evolving smoothly trial to trial within a fixed rule, the unit has learned *which rule is active*, not a decaying memory of individual trials. The phase-portrait tool of Section 12, item 2, is the wrong lens for this regime (it assumes single-trial dynamics); the criterion-trajectory plot of item 1, viewed across a block switch, is the right one.
3. Only a gain that survives *after* ablating cue_t and that varies at the single-trial resolution *within* a fixed-rule stretch is genuine short-memory dynamics, interpretable with the phase portrait and compared against M2 as originally intended.

Conflating these three is the main risk in over-interpreting a small d : a unit that is really tracking “which rule” or “how hard is this stimulus” can look, at a glance, like it is tracking “what happened last trial”, and only the ablations and the block-aware plots above tell them apart.

16.3 Consequence: what this predicts for network 2

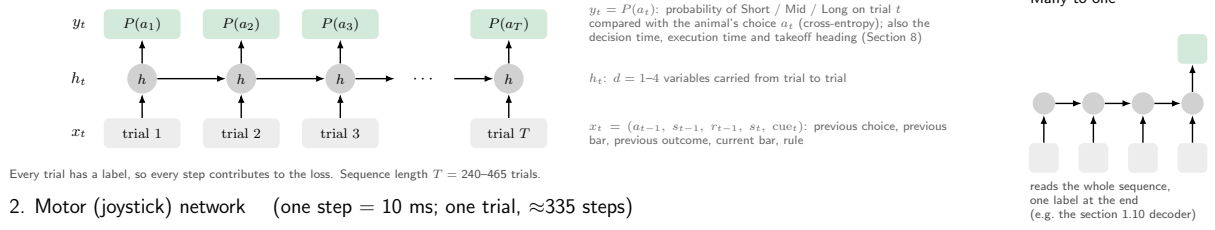
If the same-trial row of Table 22 dominates, as the weak sequential effects suggest it should, then $\mathbf{b}$ ’s value to network 2 is expected to come mainly from *within-trial decision difficulty* (D_t , how close s_t sits to a criterion) rather than from cross-trial memory. This is not a weakness of the design; it is a well-grounded, independently motivated hypothesis in its own right: harder perceptual decisions are known to produce slower, more curved, more hesitant reaches (a documented effect of decision difficulty and confidence on reach kinematics), and that effect requires no trial-to-trial memory at all, only a within-trial representation of how ambiguous the current stimulus was. Two consequences follow directly.

- The first nested control of Section 15 ($\mathbf{b}$ versus nothing) is *not* expected to come back null merely because sequential effects are weak. D_t is available every trial regardless of memory, and if the motor network uses it at all, that control will show an effect.
- The second nested control ($\mathbf{b}$ versus the raw scalar D_t) is the one that actually tests for a contribution from the other two rows of Table 22 (rule identity, single-trial history) beyond plain perceptual difficulty. Given how weak the sequential effects are, this second control is predicted

to show little or no further gain in three-category-only sessions, and a larger one in **alternate** sessions right after a block switch, where rule identity is briefly the dominant thing h_t carries.

Many-to-many: every step gets an input x_t and produces an output y_t ; the loss is summed over the steps that carry a label

1. Trial-level tiny RNN (one step = one trial; the sequence is a whole session)



2. Motor (joystick) network (one step = 10 ms; one trial, ≈ 335 steps)

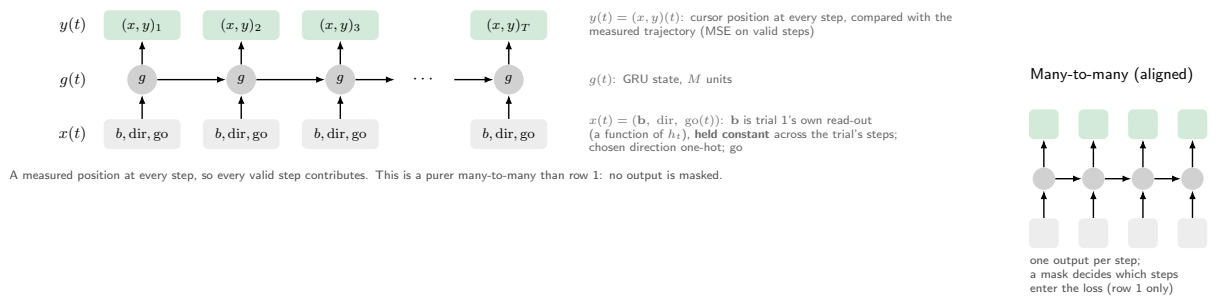


Figure 9: The two networks unrolled in time, with what x_t and y_t are in each. Row 1 is network 1 (one step per trial, every step labelled). Row 2 is the motor (joystick) network (one step per 10 ms); it has a measured position at every step, so none of its outputs are masked, unlike row 1's optional timing heads. Right: the two topologies contrasted.

16.4 Changes to the export and to the task code

Table 23: What network 2 needs from the RNN-dataset notebook and from the task.

Where	Change
Dataset notebook, labels	add the chosen category (ChosenTarget -1 , -1 if no response), the number of categories per trial, and the takeoff heading in degrees (signed angle between the cursor direction over the first 50 ms after leaving the hold window and the chosen target's direction) — these feed network 1
Dataset notebook, b construction	run the trained (frozen) network 1 forward over every session; save h_t per trial; build b from it and s_t per Eq. (19); store as a (B, k) tensor, one row per trial, to be broadcast across that trial's steps at training time
Dataset notebook, motor input	DirectionChosen as a one-hot tensor $(B, T, 4)$, on from go; a go indicator (B, T)
Dataset notebook, motor target	use the exported position divided by 406 px as the target; keep velocity as a diagnostic only; add the hold RMS in pixels to the metadata
Dataset notebook, filters	Attempt = 1; old-schema sessions get three categories
Dataset notebook, split	by session, as in Section 10.2, replacing the single chronological test session
CenterOutTask.m	export the category at each of the four positions (CatAtRight , CatAtUp , CatAtLeft , CatAtDown ; 0 = no target), which is already in memory in layoutForTrial . This is not consumed by either network as specified, but is a prerequisite for any future extension that has network 1 pick a direction from the colours instead of taking DirectionChosen as a given

A Model skeleton in PyTorch

The choice loss is $-(\text{mask} * \log \text{probs}[\text{a_obs}]).\text{sum}() / \text{mask}.\text{sum}()$; the three timing losses are Gaussian negative log-likelihoods of $\log_dt$, $\log_et$ and $\log_th$ given the returned means and spreads, with the same mask, weighted by λ_{DT} , λ_{ET} and λ_{θ} . For the feedback-inferred version the two cue_t channels of $\mathbf{x}$ are set to zero; for the choice-only model the second output is ignored.

```
import torch, torch.nn as nn, torch.nn.functional as F

class TinyRNN(nn.Module):
    def __init__(self, k=9, d=1, link="logistic"):
        super().__init__()
        self.cell = nn.GRUCell(k, d)
        self.h0 = nn.Parameter(torch.zeros(d))
        self.w_g = nn.Linear(d, 1) # x_hat = s_t + w_g.h + b_g
        self.w_c = nn.Linear(d, 2) # c_1, delta (c_2 = c_1 + softplus(delta))
        self.log_beta = nn.Parameter(torch.zeros(1))
        self.lapse_raw = nn.Parameter(torch.tensor(-3.0))
        self.link = link
        # timing heads (Sec. Timing observables): each is bias + kappa * D + w.h
        self.w_dt, self.w_et, self.w_th = nn.Linear(d, 1), nn.Linear(d, 1), nn.Linear(d, 1)
        self.kappa = nn.Parameter(torch.zeros(3)) # DT, ET, heading spread
        self.log_sd = nn.Parameter(torch.zeros(2)) # sigma_DT, sigma_ET

    def cdf(self, u):
        if self.link == "logistic":
            return torch.sigmoid(u)
        return 0.5 * (1 + torch.erf(u / 2 ** 0.5)) # cumulative Gaussian

    def forward(self, x, s): # x: (T, B, k), s: (T, B)
        T, B, _ = x.shape
        h = self.h0.expand(B, -1)
        out, mu_dt, mu_et, log_sd_th = [], [], [], []
        for t in range(T):
            h = self.cell(x[t], h)
            xhat = s[t] + self.w_g(h).squeeze(-1)
            c = self.w_c(h)
            c1 = c[:, 0]
            c2 = c1 + F.softplus(c[:, 1])
            beta = self.log_beta.exp()
            lam = 0.1 * torch.sigmoid(self.lapse_raw)
            p1 = (1 - lam) * self.cdf(beta * (xhat - c1)) + lam / 2 # P(a >= 1)
            p2 = (1 - lam) * self.cdf(beta * (xhat - c2)) + lam / 2 # P(a >= 2)
            probs = torch.stack([1 - p1, p1 - p2, p2], dim=-1).clamp_min(1e-6)
            out.append(probs)
            D = -torch.minimum((xhat - c1).abs(), (xhat - c2).abs()) # Eq. (dt-D)
            mu_dt.append(self.w_dt(h).squeeze(-1) + self.kappa[0] * D) # log DT mean
            mu_et.append(self.w_et(h).squeeze(-1) + self.kappa[1] * D) # log ET mean
            log_sd_th.append(self.w_th(h).squeeze(-1) + self.kappa[2] * D) # log heading sd
        return torch.stack(out), torch.stack(mu_dt), torch.stack(mu_et), torch.stack(log_sd_th)
```

Motor network: building $\mathbf{b}$ from the frozen network 1

This is the two-stage regime (Section 15): TinyRNN is trained and frozen first; `trial_b` below is run once per session to produce one $\mathbf{b}$ vector per trial, which MotorNet then takes as a *constant* input, broadcast across every 10 ms step of that trial.

```
@torch.no_grad()
def trial_b(tiny_rnn, x, s, W_b):
    # x, s: one session, shapes (T, 1, k) and (T, 1). Returns b: (T, k_bottleneck).
    T = x.shape[0]
    h = tiny_rnn.h0.expand(1, -1)
    hs = []
```

```

for t in range(T):
    h = tiny_rnn.cell(x[t], h)
    hs.append(h.squeeze(0))
h_seq = torch.stack(hs) # (T, d)
return W_b(torch.cat([h_seq, s], dim=-1)) # (T, k_bottleneck), Eq. (mot-b)

class MotorNet(nn.Module):
    def __init__(self, k_bottleneck=4, M=32):
        super().__init__()
        self.cell = nn.GRUCell(k_bottleneck + 4 + 1, M) # b, dir one-hot, go
        self.g0 = nn.Parameter(torch.zeros(M))
        self.w_p = nn.Linear(M, 2) # (x, y), Eq. (mot-p)

    def forward(self, b, dir_onehot, go): # b: (k,) constant this trial; dir/go: (T, .)
        T = go.shape[0]
        g = self.g0.unsqueeze(0)
        pos = []
        for t in range(T):
            x_t = torch.cat([b.unsqueeze(0), dir_onehot[t:t+1], go[t:t+1, None]], dim=-1)
            g = self.cell(x_t, g)
            pos.append(self.w_p(g).squeeze(0))
        return torch.stack(pos) # (T, 2)

```